

Présentation Quandela

El Houdaigui Karim

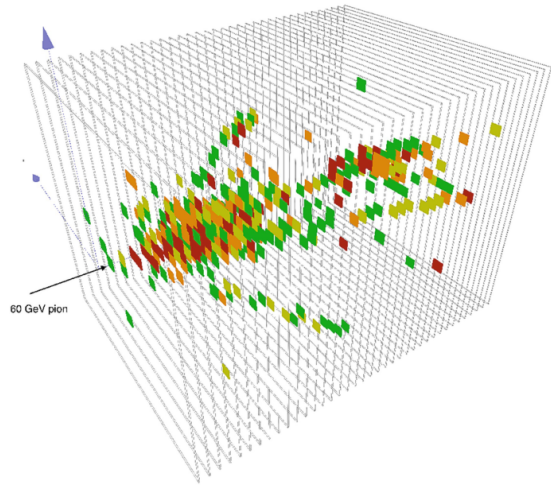
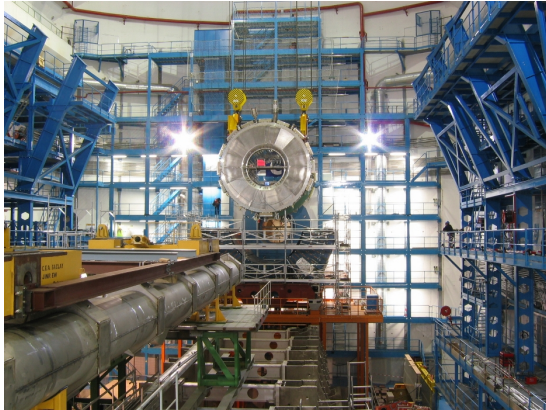
Candidature au poste de

Support Engineer (Quantum Algorithms)

Algorithme quantique photonique pour la génération d'images de calorimètres

Le projet s'inscrit à l'interface entre la **physique des hautes énergies** et la **photonique quantique**. En physique nucléaire expérimentale, les **calorimètres** sont des détecteurs utilisés pour mesurer les dépôts d'énergie laissés par des particules issues de collisions. Chaque événement produit une cascade de particules secondaires (*shower*), et la répartition de l'énergie déposée dans les cellules du détecteur forme une **image bidimensionnelle** :

Calorimeter Shower Image



Ces images servent à :

- identifier le type de particule incidente (électron, pion, photon, etc.) ;
- entraîner des modèles de reconstruction et de simulation.

Actuellement, ces distributions d'énergie sont souvent reproduites à l'aide de **modèles génératifs classiques** comme les *Generative Adversarial Networks (GANs)*. L'objectif ici est de proposer une **approche quantique photonique**, fondée sur la technologie de **Quandela (Perceval)**, pour générer ces images de manière intrinsèquement probabiliste et plus compacte en paramètres.

L'idée directrice est de **traduire la distribution d'énergie mesurée dans le calorimètre en une distribution photonique** issue d'un **circuit interférométrique**.

2. Modélisation du problème

2.1. Objectif théorique

On cherche à contrôler la répartition des photons entre m modes optiques de sortie. Chaque mode représente une région condensée du calorimètre. Le comportement global du circuit est décrit par une **transformation unitaire** $U \in U(m)$ appliquée sur le vecteur d'amplitudes photoniques :

$$\mathbf{a}_{\text{out}} = U(\boldsymbol{\theta}) \mathbf{a}_{\text{in}} \quad (1)$$

où :

- $\mathbf{a}_{\text{in}} = (a_1, a_2, \dots, a_m)^T$ est le vecteur d'amplitudes d'entrée (modes injectés) ;
- $\mathbf{a}_{\text{out}}$ est le vecteur d'amplitudes détectées en sortie ;
- $U(\boldsymbol{\theta})$ est la matrice unitaire dépendant de paramètres optiques $\boldsymbol{\theta}$ (phases, couplages, etc.).

Le but est d'ajuster les paramètres $\boldsymbol{\theta}$ pour que la distribution de détection photonique $P_{U(\boldsymbol{\theta})}(n_1, \dots, n_m)$ reproduise statistiquement la distribution d'énergie observée dans les images réelles.

2.2. Implémentation physique : le schéma de Clements

Pour construire une matrice unitaire U en laboratoire, on utilise un réseau de **beam splitters (BS)** et de **phase shifters (PS)**. Le **schéma de Clements** permet de décomposer toute matrice $U \in U(m)$ en une succession finie et déterministe de ces composants :

$$\frac{m(m-1)}{2} \text{ BS} \quad \text{et} \quad \frac{m(m-1)}{2} \text{ PS.}$$

Cette architecture garantit la **complétude unitaire** : toute distribution photonique réalisable peut être implémentée par un circuit Clements correctement paramétré. C'est la raison pour laquelle cette topologie est utilisée par les plateformes photoniques programmables de Quandela.

Le schéma de Clements assure que chaque ensemble de paramètres optiques $\boldsymbol{\theta}$ correspond à une transformation unitaire unique, et donc à une distribution photonique bien définie.

3. Passage des images au format vectoriel

3.1. Image physique

Une image expérimentale du calorimètre est une matrice d'énergie $E(x, y)$, par exemple 30×30 :

$$\text{Image} = \begin{bmatrix} 0.0 & 0.2 & 0.1 & \dots \\ 0.4 & 1.0 & 0.8 & \dots \\ \vdots & \vdots & \vdots & \ddots \end{bmatrix}$$

Cette matrice est aplatie en un vecteur colonne :

$$\mathbf{E} = (E_1, E_2, \dots, E_{900})^T.$$

3.2. Réduction de dimension

On réduit la dimension de $\mathbf{E}$ à m composantes principales via une **PCA** ou un **autoencodeur** :

$$\mathbf{E} \longrightarrow \mathbf{E}' = (E'_1, E'_2, E'_3, E'_4)$$

Chaque composante E'_i représente une région condensée (ou un motif global) de la shower.

3.3. Correspondance image – modes photoniques

On établit alors la correspondance :

$$\begin{aligned} E'_1 &\leftrightarrow M_1 \\ E'_2 &\leftrightarrow M_2 \\ E'_3 &\leftrightarrow M_3 \\ E'_4 &\leftrightarrow M_4 \end{aligned} \quad \Longleftrightarrow \quad \mathbf{E}' = (E'_1, E'_2, E'_3, E'_4) \leftrightarrow \mathbf{n} = (n_1, n_2, n_3, n_4)$$

où n_i est le nombre de photons détectés dans le mode M_i . Ainsi, une image condensée correspond directement à une configuration photonique de détection.

4. Distribution photonique = image énergétique

Dans un circuit photonique, on peut calculer la probabilité de détecter un certain nombre de photons dans chaque mode :

$$P_{U(\boldsymbol{\theta})}(n_1, n_2, \dots, n_m)$$

C'est une **distribution de probabilité** sur les configurations de photons $(n_1, n_2, \dots, n_m)$.

L'idée est de faire correspondre **cette distribution photonique** à la **distribution d'énergie observée dans les images réelles de calorimètres**.

Les n_i (nombres de photons détectés) jouent le rôle des énergies E'_i .

L'interféromètre, via sa matrice unitaire $U(\boldsymbol{\theta})$, est le **générateur** qui crée ces distributions.

5.L'apprentissage

L'objectif est d'ajuster les paramètres optiques $\boldsymbol{\theta}$ (phases et couplages du circuit) pour que la distribution photonique simulée reproduise la distribution expérimentale :

$$P_{U(\boldsymbol{\theta})}(\mathbf{n}) \approx P_{\text{data}}(\mathbf{E}')$$

avec :

$$\begin{cases} P_{\text{data}}(\mathbf{E}') : \text{les vraies distributions extraites des images du calorimètre,} \\ P_{U(\boldsymbol{\theta})}(\mathbf{n}) : \text{les distributions générées par le circuit photonique.} \end{cases}$$

On cherche donc à **entraîner la transformation unitaire** $U(\boldsymbol{\theta})$ (en modifiant les phases optiques) jusqu'à ce que les deux distributions soient presque identiques :

$$P_{U(\boldsymbol{\theta})}(\mathbf{n}) \simeq P_{\text{data}}(\mathbf{E}')$$

Autrement dit, le réseau photonique agit comme un **modèle génératif quantique** : il apprend à produire des configurations photoniques dont les statistiques reproduisent les motifs énergétiques caractéristiques des showers observés expérimentalement.

4. Implémentation dans Perceval

4.1. Construction du circuit

Exemple Python

```
import perceval as pcvl
from perceval.components import GenericInterferometer, PS

m = 4 # nombre de modes (pixels condensés)
circuit = GenericInterferometer(m, structure="clements")

# Ajout de phase-shifters paramétriques
for i in range(m):
    circuit.add(i, PS(phi=pcvl.P(f"phi{i}")))
```

Chaque ϕ_i est un paramètre optimisable contrôlant la phase optique appliquée au mode i .

4.2. État d'entrée

Exemple Python

```
input_state = pcvl.BasicState([1, 0, 1, 0])
```

Deux photons sont injectés dans les modes M_1 et M_3 . L'état quantique correspondant est :

$$|\psi_{\text{in}}\rangle = |1, 0, 1, 0\rangle.$$

4.3. Simulation et distribution de sortie

Exemple Python

```
proc = pcvl.Processor("SLOS", circuit)
proc.with_input(input_state)
results = proc.sample(1000)
```

Le processeur simule 1000 détections aléatoires selon la distribution :

$$P_{U(\boldsymbol{\theta})}(n_1, n_2, n_3, n_4)$$

Chaque ligne de **results** correspond à une configuration de détection possible :

$$[1, 0, 1, 0], \quad [0, 1, 1, 0], \quad [1, 1, 0, 0], \dots$$

La distribution empirique $\hat{P}_U$ est ensuite comparée à la distribution réelle P_{data} extraite des images.

À partir des 1000 tirages obtenus à l'étape précédente, on peut construire la distribution empirique $\hat{P}_U$, c'est-à-dire la fréquence observée de chaque configuration photonique (n_1, n_2, n_3, n_4) .

Exemple numérique :

Sur 1000 échantillons simulés, on observe :

$$\begin{aligned} [1, 0, 1, 0] & \text{ apparaît 410 fois,} \\ [0, 1, 1, 0] & \text{ apparaît 320 fois,} \\ [1, 1, 0, 0] & \text{ apparaît 270 fois.} \end{aligned}$$

La distribution empirique correspondante est alors :

$$\hat{P}_U([1, 0, 1, 0]) = 0.41, \quad \hat{P}_U([0, 1, 1, 0]) = 0.32, \quad \hat{P}_U([1, 1, 0, 0]) = 0.27.$$

Définition formelle :

Si l'on note $N(\mathbf{n})$ le nombre de fois où la configuration $\mathbf{n}$ est observée parmi les échantillons, alors :

$$\hat{P}_U(\mathbf{n}) = \frac{N(\mathbf{n})}{\sum_{\mathbf{n}'} N(\mathbf{n}')} = \frac{N(\mathbf{n})}{1000}.$$

Cette distribution empirique $\hat{P}_U$ dépend des paramètres du circuit (les phases $\boldsymbol{\theta} = (\phi_0, \phi_1, \dots)$). L'objectif de l'apprentissage est de rendre cette distribution aussi proche que possible de la distribution cible $\hat{P}_{\text{data}}$, issue du **dataset des images de calorimètre**, après :

1. réduction dimensionnelle,
2. quantification en configurations photoniques.

Autrement dit, on ajuste les phases $\boldsymbol{\theta}$ du circuit pour que la loi de probabilité générée par l'interféromètre photonique reproduise celle observée expérimentalement.

5. Entraînement variationnel

On minimise une fonction coût mesurant la divergence entre les distributions réelles et générées. Une mesure classique est la divergence de Kullback–Leibler :

$$C(\boldsymbol{\theta}) = D_{KL}(P_{\text{data}} \parallel P_{U(\boldsymbol{\theta})}) = \sum_{\mathbf{n}} P_{\text{data}}(\mathbf{n}) \log \frac{P_{\text{data}}(\mathbf{n})}{P_{U(\boldsymbol{\theta})}(\mathbf{n})}$$

L’optimisation des ϕ_i peut être effectuée par un algorithme classique. Le circuit photonique agit ainsi comme un **générateur variationnel quantique (VQC)** entraîné à reproduire les statistiques d’images de calorimètres.

6. Interprétation physique et intérêt

- **Nature probabiliste intrinsèque** : la génération d’images de showers est par essence aléatoire. Les circuits photoniques produisent naturellement des distributions de probabilité.
- **Corrélations quantiques** : les interférences entre photons créent des corrélations non classiques analogues aux structures collectives dans les gerbes.
- **Efficacité paramétrique** : quelques dizaines de phases optiques suffisent à générer des distributions riches, là où un GAN classique nécessite des millions de paramètres.

En conclusion, ce modèle photonique quantique offre une approche élégante et physiquement cohérente pour la **génération variationnelle d’images de calorimètres**, où les motifs lumineux détectés traduisent directement les distributions d’énergie observées expérimentalement.

Partie II – Tickets utilisateurs Quandela

Ticket 1 – Décomposition de Clements d’une matrice unitaire

Question du client. *Je me demandais s’il existe une implémentation de la décomposition de Clements d’une matrice unitaire dans Perceval. Je sais qu’on peut utiliser le circuit optimiser avec un interféromètre générique pour approximer la décomposition, mais existe-t-il une version déterministe (exacte) ?*

Contexte. Le client possède une matrice unitaire $U \in U(m)$ et souhaite en extraire directement les paramètres optiques (angles des *beam splitters* et des *phase shifters*) selon le **schéma de Clements**, sans passer par une étape d’optimisation numérique.

En optique quantique linéaire, on ne programme pas directement une matrice U sur la puce : on la réalise à l'aide d'un réseau d'interféromètres composé de composants physiques :

- des **Beam Splitters (BS)** : qui couplent deux modes optiques ;
- des **Phase Shifters (PS)** : qui ajoutent un déphasage contrôlable sur un mode donné.

1. Le schéma de Clements (méthode déterministe)

Le schéma de **Clements** (Optica, 3, 1460, 2016) est une méthode analytique permettant de décomposer toute matrice unitaire U en un réseau rectangulaire de $\frac{m(m-1)}{2}$ *beam splitters* et *phase shifters*. Chaque composant correspond à une transformation 2×2 du type :

$$T(\theta, \phi) = \begin{bmatrix} \cos \theta & -e^{i\phi} \sin \theta \\ e^{-i\phi} \sin \theta & \cos \theta \end{bmatrix}$$

et la matrice complète est le produit ordonné de ces transformations :

$$U = \prod_{k=1}^N T_k(\theta_k, \phi_k).$$

Ainsi, pour une unitaire donnée, on peut en principe déterminer analytiquement tous les angles θ_k et ϕ_k .

2. Exemple simple : la porte de Hadamard

Considérons la matrice unitaire :

$$U = \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{bmatrix},$$

qui correspond à une **porte de Hadamard optique**, équivalente à un *beam splitter* 50/50.

La décomposition de Clements (ou Reck) nous donne directement :

$$\theta = \frac{\pi}{4}, \quad \phi = 0,$$

soit un simple coupleur à 50/50. Aucune optimisation numérique n'est nécessaire.

3. Approche actuelle dans Perceval (optimisation numérique)

Dans la version publique de **Perceval**, la fonction de décomposition analytique n'est pas encore disponible. En revanche, on peut construire un interféromètre paramétrique de type Clements et ajuster ses paramètres par optimisation numérique :

Exemple Python

```
from perceval.components import GenericInterferometer
from perceval.algorithm import optimize_unitary

# Matrice cible : porte de Hadamard
U_target = [
    [1/2**0.5, 1/2**0.5],
    [1/2**0.5, -1/2**0.5]
]

# Création d'un interféromètre 2x2 de structure Clements
interf = GenericInterferometer(2, structure="clements")

# Recherche numérique des paramètres phi_i, theta_i
optimize_unitary(interf, U_target)
```

Cette fonction ajuste les angles $\{\theta_i, \phi_i\}$ afin de minimiser la distance :

$$\min_{\theta_i, \phi_i} \|U_{\text{sim}}(\theta_i, \phi_i) - U_{\text{target}}\|,$$

et retourne une approximation satisfaisant :

$$\|U_{\text{sim}} - U_{\text{target}}\| < \varepsilon,$$

où ε est une tolérance numérique.

Résultat de l'optimisation numérique

Après exécution de la fonction `optimize_unitary`, on peut interroger l'objet retourné `result` :

Résultat de l'optimisation

```
print(result)
```

Sortie typique :

OptimizationResult:

```
success: True
loss: 3.2e-09
nfev: 45
parameters: {'BS_0_theta': 0.7854, 'BS_0_phi': 0.0,
              'PS_0_phi': 0.0, 'PS_1_phi': 3.1416}
```

L'objet `OptimizationResult` contient :

- `success` : `True` — l'optimisation a convergé ;
- `loss` : `3.2e-09` — la distance entre la matrice simulée et la matrice cible est très faible ;
- `parameters` — dictionnaire des valeurs optimisées pour les angles optiques.

L'interféromètre `interf` est désormais configuré avec ces paramètres. En affichant sa description :

Description de l'interféromètre configuré

```
interf.describe()
```

On obtient :

```
GenericInterferometer(
  structure="clements",
  parameters={
    'BS_0_theta': 0.785398,    # /4
    'BS_0_phi': 0.0,
    'PS_0_phi': 0.0,
    'PS_1_phi': 3.14159
  }
)
```

La matrice unitaire interne correspond numériquement à la matrice cible :

Matrice unitaire simulée

```
print(interf.compute_unitary())
```

```
[[ 0.70710678+0.j  0.70710678+0.j]
 [ 0.70710678+0.j -0.70710678+0.j]]
```

Cette matrice correspond bien à la **porte de Hadamard** :

$$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}.$$

Ainsi, la fonction `optimize_unitary` permet de déterminer automatiquement les angles (θ_i, ϕ_i) de l'interféromètre pour reproduire une transformation unitaire cible, et de configurer directement le réseau de Clements correspondant.

4. Réponse finale à fournir au client

Perceval implémente bien l'architecture de Clements, permettant de réaliser toute matrice unitaire sur m modes optiques via des *beam splitters* et *phase shifters*. Cependant, il n'existe pas encore de fonction publique de décomposition déterministe $U \mapsto \{\theta_i, \phi_i\}$.

Actuellement, la démarche consiste à :

1. Créer un interféromètre de type Clements avec `GenericInterferometer(m, structure="clements");`
2. Utiliser `optimize_unitary` pour ajuster les phases et couplages de manière à approximer la matrice cible.

Si une décomposition exacte est nécessaire, l'utilisateur peut implémenter l'algorithme de Clements (Optica, 2016) et injecter les angles obtenus dans le circuit Perceval.

,

Ticket 2

Un client travaille avec **Qiskit** et souhaite exécuter son programme QASM sur un **ordinateur quantique photonique** de **Quandela**. Pour cela, il utilise le **QiskitConverter** de Perceval :

Exemple Python

```
from qiskit import QuantumCircuit
from perceval.converters import QiskitConverter

q_circuit = QuantumCircuit.from_qasm_str(qasm_code)
converter = QiskitConverter()
p_processor = converter.convert(q_circuit)
```

Le résultat est un objet `Processor`, alors que le QPU attend un objet `Circuit`. Pourquoi ? Et comment l'utiliser correctement ?

2. Les trois couches fondamentales de Perceval

Perceval sépare clairement trois niveaux d'abstraction :

A. Circuit — Schéma optique pur

C'est le **plan optique**, définissant la topologie du réseau photonique :

- nombre de modes optiques ;
- emplacements des *beam splitters* (BS) et *phase shifters* (PS) ;
- structure du réseau (Reck, Clements, MZI...).

Important : un `Circuit` ne contient ni sources ni détecteurs. C'est ce que le QPU réel exécute directement.

Exemple Python

```
import perceval as pcvl
circuit = pcvl.Circuit(4)
```

B. Processor — Expérience photonique complète

Un `Processor` est un **laboratoire virtuel** :

- contient un circuit optique ;
- spécifie les sources et détecteurs ;
- peut inclure du *feed-forward* logique.

Exemple Python

```
proc = pcvl.Processor("SLOS", circuit)
proc.with_input(pcvl.BasicState([1,0,1,0]))
proc.sample_count(1000)
```

C. RemoteProcessor — Exécution sur QPU réel

Le `RemoteProcessor` envoie le circuit pur au QPU Quandela :

Exemple Python

```
from perceval.backends import RemoteProcessor

rproc = RemoteProcessor("Quandela_QPU_Lucas", circuit)
rproc.with_input(pcvl.BasicState([1,0,1,0]))
res = rproc.sample_count(1000)
```

Le matériel (sources et détecteurs) est déjà intégré au QPU : seul le schéma optique est nécessaire.

4. Pourquoi le `QiskitConverter` renvoie un `Processor`

Le `QiskitConverter` traduit les portes Qiskit (logique des qubits) en équivalents photoniques (modes, BS, PS), et crée une expérience complète pour simulation.

Ainsi, il renvoie un `Processor`, pas un `Circuit`, car il modélise déjà les sources et détecteurs virtuels.

Solution

Étape 1 — Extraire le circuit du `Processor`

Extraction du circuit pur

```
circuit = p_processor.circuit
```

L'attribut `.circuit` contient la **topologie photonique pure** (le plan optique que le QPU exécutera).

Étape 2 — Configurer et lancer sur le QPU réel

Exécution sur QPU Quandela

```
from perceval.backends import RemoteProcessor
import perceval as pcvl

# Initialisation du QPU distant
rproc = RemoteProcessor("Quandela_QPU_name", circuit)

# Choisir l'état d'entrée (ex. 1 photon sur mode 0 et 1 photon sur mode 2)
rproc.with_input(pcvl.BasicState([1,0,1,0]))

# Exécution (par ex. 1000 échantillons)
res = rproc.sample_count(1000)

print(res)
```

Cette configuration envoie le `Circuit` au QPU Quandela distant. Les sources et détecteurs étant physiques, seul le schéma optique est transmis.

Étape 3 — Simulation locale (optionnelle)

Exécution locale simulée

```
p_processor.with_input(pcvl.BasicState([1,0,1,0]))
res = p_processor.sample_count(1000)
```

Cette méthode exécute le même circuit en mode **simulation logicielle**, via le backend local de Perceval (`Processor`). Elle permet de valider le comportement du circuit avant l'envoi au QPU réel.

Phrase synthèse

On récupère le schéma optique pur avec `p_processor.circuit`, on l'envoie au QPU via un `RemoteProcessor`, et si besoin on teste le même montage localement via le `Processor` pour simuler la physique photonique.

Résumé

Le `QiskitConverter` renvoie un `Processor`, car il modélise une expérience photonique complète pour la simulation. Le QPU, lui, n'a besoin que du *plan optique pur* : un `Circuit`. On extrait donc le circuit via `p_processor.circuit`, puis on l'envoie au `RemoteProcessor` pour l'exécution réelle.

1 Feed-forward adaptatif dans Perceval : exemple complet

Objectif du ticket

L'objectif est d'ajouter deux mesures photoniques successives (deux étapes adaptatives), où la seconde, qui contrôle le paramètre `phi2`, dépend des résultats de **deux mesures précédentes** : le nombre de photons détectés sur les modes 0 et 1.

Autrement dit :

Si un photon est détecté sur le mode 0 et un photon sur le mode 1, alors appliquer une valeur particulière à `phi2`.

Partie 1 — Lecture du code initial ligne par ligne

Initialisation du processeur

```
import perceval as pcvl
p = pcvl.Processor("SLOS", 4)
```

Explication :

- `Processor("SLOS", 4)` crée un processeur photonique simulé à 4 modes optiques.
- SLOS (Stochastic LO Simulation) est le moteur de simulation de Perceval.
- Chaque mode représente une voie optique où circulent les photons.

Définition des paramètres symboliques

```
phi1 = pcvl.P("phi1")
phi2 = pcvl.P("phi2")
```

Explication :

- `pcvl.P("phi1")` crée un paramètre symbolique : une variable que l'on pourra modifier via un `FFConfigurator`.

Étape 1 — Première mesure adaptative

Déclaration de la première configuration adaptative

```
c_1 = pcvl.FFConfigurator(  
    1,                                # lit 1 mode classique  
    offset=1,                         # agit sur la partie suivante du circuit  
    controlled_circuit=pcvl.PS(phi1),  
    default_config={"phi1": 0},  
    name="U_1"  
)  
c_1.add_configuration([1], {"phi1": 1.23})
```

Détails :

- `FFConfigurator` : composant de feed-forward.
- `m=1` : lit le résultat d'un seul mode.
- `offset=1` : agit juste après la mesure.
- `controlled_circuit` : un phase-shifter (retard de phase).
- `add_configuration([1], {"phi1": 1.23})` : si un photon est détecté, alors $\phi_1 = 1.23$.

Ajout au processeur

```
p.add(0, pcvl.Detector.pnr())  
p.add(0, c_1)
```

Explication :

- Sur le mode 0, on ajoute un détecteur PNR (Photon-Number-Resolving).
- Juste après, on place le configurator `c_1` : il lit le résultat et ajuste ϕ_1 .

Étape 2 — Deuxième mesure adaptative

Déclaration de la deuxième configuration

```
c_2 = pcvl.FFConfigurator(  
    1, offset=1,  
    controlled_circuit=pcvl.PS(phi2),  
    default_config={"phi2": 0},  
    name="U_2"  
)  
c_2.add_configuration([1], {"phi2": 1.23})
```

Même logique que la première, mais ici le `FFConfigurator` agit sur le mode 1.

Problème : dépendance entre deux mesures

L'utilisateur veut que `c_2` dépende de deux modes (0 et 1), c'est-à-dire que la configuration soit conditionnée par le motif `[1, 1]`. Or, `c_2` lit actuellement un seul mode (`m=1`).

Partie 2 — Correction complète

Code corrigé complet et commenté

```
import perceval as pcvl
import numpy as np

p = pcvl.Processor("SLOS", 4)

phi1 = pcvl.P("phi1")
phi2 = pcvl.P("phi2")

# --- Première mesure adaptative
c_1 = pcvl.FFConfigurator(
    m=1,
    offset=1,
    controlled_circuit=pcvl.PS(phi1),
    default_config={"phi1": 0},
    name="U_1"
)
c_1.add_configuration([1], {"phi1": 1.23})
p.add(0, pcvl.Detector.pnr())
p.add(0, c_1)

# --- Deuxième mesure adaptative dépendant de 2 modes
c_2 = pcvl.FFConfigurator(
    m=2,
    offset=0,
    controlled_circuit=pcvl.PS(phi2),
    default_config={"phi2": 0},
    name="U_2"
)
c_2.add_configuration([0, 1], {"phi2": np.pi/4})
c_2.add_configuration([1, 1], {"phi2": np.pi/2})
p.add(1, pcvl.Detector.pnr())
p.add(1, c_2)

pcvl.pdisplay(p)
```

Partie 3 — Lecture conceptuelle

Élément	Rôle
<code>Detector.pnr()</code>	Détecte le nombre de photons sur un mode.
Mode classique	Mode dont le photon a été mesuré : il contient désormais une valeur entière classique.
<code>FFConfigurator</code>	Lit ces valeurs et applique la configuration correspondante du circuit.
<code>m</code>	Nombre de modes classiques lus.
<code>offset</code>	Décalage : à partir de quel mode commence la lecture.
<code>add.configuration</code>	Règle : motif de mesure $\rightarrow$ configuration de paramètres.

Partie 4 — Réponse à la question finale

Question : *Can I access the number of detected photons in PNRs anywhere ?*

Réponse :

- Les résultats de détection ne sont pas accessibles directement comme variables globales.
- Ils sont transmis automatiquement aux `FFConfigurator` ou `FFCircuitProvider`.
- Pour dépendre de plusieurs mesures, on fixe simplement `m = nombre de modes` et on déclare les motifs de détection associés : $[n_0, n_1, \dots]$.
- Le feed-forward est donc entièrement **déclaratif** : on décrit les règles avant la simulation.

Bilan

- Les détecteurs PNR convertissent les états quantiques en valeurs classiques.
- Les modes ainsi mesurés deviennent lisibles par les `FFConfigurator`.
- Chaque configurator applique une configuration en fonction du motif de détection.
- Pour combiner plusieurs mesures, on lit plusieurs modes (`m=2`, `m=3`, etc.).
- Il n'existe pas d'accès direct aux résultats de mesure : tout est défini avant l'exécution.